Bridge Backend Engineering Handoff

Student talent marketplace | Python API service and screen-to-backend handoff

Purpose and scope

This handoff translates the visible Bridge mobile prototype into backend work areas. The screens depict students presenting skills and portfolios, browsing paid tasks, submitting proposals, and communicating with task owners. Task owners can post work, review applicants, view talent profiles, manage matches, and exchange messages. The screens are evidence of intended interactions; endpoint names, persistence rules, and security behavior below are implementation recommendations unless explicitly marked as a selected technical decision.

Evidence boundary: Screen descriptions reflect the Google Stitch project. Proposed behavior is identified as such where the UI does not define the rule. No payment flow, ratings calculation, verification workflow, or operational policy is confirmed by the screens.

Selected backend stack

Layer	Selection	Responsibility
API service	Python with FastAPI	Separate, versioned HTTPS REST API between mobile clients and backend domain logic.
Request models	Pydantic	Parse and validate API input and produce OpenAPI schemas.
Database	PostgreSQL	Transactional persistence for profiles, tasks, applications, conversations, and messages.
Database access	SQLAlchemy 2.x	ORM and transaction/session management.
Schema migrations	Alembic	Versioned database schema changes.
Authentication	Supabase Auth	Email/password and Google sign-in; clients send Supabase access JWTs to the API.

The mobile app calls FastAPI; it does not access marketplace tables directly. The API verifies the Supabase JWT signature, issuer, and expiry using Supabase's published signing keys (JWKS), then applies object-level authorization. Cache signing keys according to library/provider guidance and refresh them safely for key rotation. Do not implement JWT cryptography manually.

Use SQLAlchemy transaction boundaries for multi-row state changes, and PostgreSQL constraints for uniqueness and referential integrity. If the API connects with a privileged database role that bypasses row-level security, RLS is not a substitute for API authorization. Use a least-privileged database role; treat RLS as defense in depth only when the request identity is safely propagated to the database.

Keep one API service for the initial scope. Add a worker, queue, realtime transport, or additional service only when a confirmed product capability requires it. The separate API service requirement is fulfilled by the FastAPI boundary; the prototype does not establish a microservice topology.

Screen inventory

Screen	Visible content and interaction	Backend implication
Splash screen	Bridge branding and loading state.	No domain operation shown; startup may later fetch session/config.
Prototype launch	"Double click to open" prototype card.	Design navigation only; not a product requirement.
Role selection	Student or Freelancer choice; Continue.	Persist selected account mode or use it to route onboarding.

Screen	Visible content and interaction	Backend implication
Login / signup	Email, password, Login, Google continuation, sign-up link.	Authenticate through Supabase Auth; signup and recovery details are not shown.
Student onboarding	Full name, skill tags, experience level, weekly availability, portfolio link; save/continue.	Profile create/update with validated skills and optional portfolio URL.
Task details	Title, image, budget, deadline, description, skills; client/task summary.	Read task detail and related client metadata.
Student home	Active task count, pending payment amount, available task cards with budget and detail link.	Dashboard summary plus paginated task discovery; metric definitions remain open.
Apply to task	Task summary, proposal text, portfolio highlights and edit selection.	Create application/proposal linked to task and applicant.
Student profile	Name, avatar, title, experience, portfolio URL, skills, edit profile.	Read/update profile and portfolio references.
Freelancer home	Post New Task; suggested student cards with skill/rating summary and profile action.	Create-task entry point and candidate recommendations/search; ranking behavior is not specified.
Matches	Accepted, pending, and alternative/rejected-looking application cards; open chat; edit action on some cards.	List relationship/application states and create/open a conversation for eligible pairs.
Post a task	Title, description, required skills, budget, deadline, draft/review control.	Create/update task listing; draft/review meaning needs product definition.
Freelancer profile	Name, rating, verified badge, rate/earnings, completion percentage, active tasks.	Read public profile and task/aggregate metrics; verification source is unspecified.
Chat	Conversation history, message composer, send control, project goal/progress card.	Fetch conversation and append messages; project goal/progress behavior needs definition.
Task applicants	Applicant profile/rating/skills with Reject and Accept actions.	List applications and transition their state with authorization and concurrency checks.

User flows

Sign in and build a student profile

The user chooses Student, signs in through Supabase Auth using email/password or Google, and completes a profile with name, skills, experience, availability, and portfolio link. The mobile client obtains the Supabase session and sends its access JWT as a bearer token to FastAPI. FastAPI verifies the token and upserts only the caller's profile. The prototype does not show email verification, password reset, or signup fields.

Discover and apply to work

Student home presents available tasks and summary metrics. The student opens task details, writes a proposal, selects portfolio highlights, and submits. Recommended application lifecycle: pending -> accepted or rejected; withdrawal, editing after submission, and resubmission are not visibly specified.

Client posts and reviews a task

Freelancer/client home offers Post New Task. The task form collects title, description, required skills, budget, and deadline. Applicants are shown with profile evidence and Accept/Reject controls. Save as draft versus publication behavior needs clarification.

Match and message

Matches groups candidates/application cards into visible status sections, and accepted matches expose Open Chat. A conversation screen shows messages and a project goal/progress card. Recommended: allow conversation access only for an accepted application or another explicitly approved relationship; enforce membership for every read/send.

Suggested API surface

Convention: Example REST paths and payload names below are proposals. Use `/v1`, JSON, a consistent error envelope, cursor pagination, and ISO-8601 UTC timestamps. FastAPI/Pydantic should define the request and response schemas and publish the OpenAPI contract.

Method and path	Purpose	Key behavior
Supabase Auth SDK: sign-in, signup, OAuth, refresh, signout	Establish and manage authentication session.	Client uses Supabase Auth; do not duplicate auth endpoints in FastAPI unless a concrete requirement needs a server-owned session layer.
`GET /v1/me` · `PATCH /v1/me/profile`	Read identity and edit profile.	Require verified bearer JWT; accept only editable fields.
`GET /v1/tasks`	Browse available tasks.	Filter by skill, budget, deadline; cursor pagination; exclude non-published tasks.
`GET /v1/tasks/{taskId}`	Task detail.	Include only public task and client summary fields.
`POST /v1/tasks` · `PATCH /v1/tasks/{taskId}`	Create/edit task.	Owner-only; draft/publish state transition must be agreed.
`POST /v1/tasks/{taskId}/applications`	Submit proposal.	One active application per user/task unless product chooses otherwise; idempotency recommended.
`GET /v1/tasks/{taskId}/applications`	Review applicants.	Task owner only; paginated applicant cards.
`PATCH /v1/applications/{applicationId}`	Accept/reject application.	Owner-only; transactional state transition; handle concurrent decisions.
`GET /v1/me/applications` · `GET /v1/me/matches`	Student status lists.	Expose only caller's applications and permitted match states.
`GET /v1/profiles/{profileId}`	Profile detail.	Return public profile fields; redact private account data.
`GET /v1/conversations` · `GET /v1/conversations/{id}/messages`	List/read conversations.	Member-only, ordered by sent time, cursor paginated.
`POST /v1/conversations/{id}/messages`	Send message.	Member-only; validate body; persist and return canonical message.

Suggested data model

Entity	Suggested fields	Notes
User	`id`, Supabase `auth_user_id`, `email` if needed, `status`, `created_at`	Auth credentials remain with Supabase Auth; enforce unique mapping to auth user ID.
Profile	`user_id`, `display_name`, `headline`, `avatar_url`, `experience_level`, `availability_hours_per_week`, `portfolio_url`, `rating_summary`, `verified_at`	Rating and verification calculation/source are unknown.
Skill	`id`, `canonical_name`	Canonical catalog supports consistent filtering.
ProfileSkill	`profile_id`, `skill_id`, proficiency or display order	Only add proficiency if product needs it; UI currently shows tags.
Task	`id`, `owner_user_id`, `title`, `description`, `budget_amount`, `currency`, `deadline_at`, `status`, timestamps	Use decimal or integer minor units plus currency; define draft/review/published states.
TaskSkill	`task_id`, `skill_id`	Many-to-many required skills.

Entity	Suggested fields	Notes
Application	`id`, `task_id`, `applicant_user_id`, `proposal`, `status`, `submitted_at`, `updated_at`	Define duplicate-application policy and enforce it in the database.
ApplicationPortfolioItem	`application_id`, `portfolio_item_id` or URL snapshot	UI lets applicant choose highlights; decide snapshot vs live profile reference.
Conversation	`id`, `task_id` nullable, `created_at`	Tie chat to accepted work if task-scoped conversation is confirmed.
ConversationMember	`conversation_id`, `user_id`, `joined_at`	Unique membership pair supports authorization.
Message	`id`, `conversation_id`, `sender_user_id`, `body`, `sent_at`, `read_at` nullable	Append-only preferred; read receipts are not clearly shown.
TaskProgress / ProjectGoal	`task_id`, goal text/status or progress fields	A progress/goal card appears in chat; exact fields and update controls need definition.

Authentication, authorization, and privacy

- Supabase Auth owns password and Google authentication. The client sends its access token to protected FastAPI routes as `Authorization: Bearer <JWT>`.
- Validate JWT signature against Supabase signing keys/JWKS and check issuer, expiry, and relevant claims with a maintained JWT library. Never trust a user ID or role supplied only in the request body.
- Resolve the token subject to the Bridge user/profile record. Enforce object-level authorization: task owner edits tasks and decides applications; applicant accesses only their applications; conversation members alone read/send messages.
- Treat Student/Freelancer as product modes, not proof of authorization. Whether a user may hold both modes remains open.
- Return public profile fields through an explicit allowlist. Do not expose email, session data, or private applicant details in public cards.
- Use a least-privileged Postgres role. If privileged credentials bypass RLS, implement and test authorization in FastAPI; use RLS only as additional defense where the database connection carries safe user context.
- Rate-limit auth-related and abuse-prone actions at an appropriate edge/API layer. Do not store Supabase secrets in the mobile app or log tokens and sensitive message contents.

Validation and consistency

Use Pydantic schemas for input shape and field constraints, with domain-level checks and PostgreSQL constraints for rules that must survive concurrent requests.

Input/action	Server-side checks to define
JWT	Signature, issuer, expiry, subject, and allowed authentication context; return generic 401 for invalid credentials.
Skill tags	Accept known skill IDs or controlled text; cap count and length; reject duplicates.
Availability	Non-negative bounded weekly hours; clarify whether required.
Portfolio URL	Allow approved schemes (normally HTTPS); length limit; do not fetch arbitrary URLs from the backend.
Task title/description	Required title and description limits; reject empty/whitespace-only values; safely render user text.
Budget/deadline	Currency and amount range; precision; deadline timezone and whether past dates are allowed.
Application	Non-empty proposal and size limit; task must be open; prevent duplicate submissions and handle concurrent close/accept.
Accept/reject	Allow only valid state transitions; idempotent retries; define whether accepting one applicant closes task or rejects others.
Message	Non-empty bounded body; safe display encoding; enforce membership at send time.

Open questions before implementation

1. Are Student and Freelancer roles exclusive, switchable modes, or labels for two user types? Can one account act as both applicant and task owner?
2. Does "Continue with Google" create an account, sign in, or both? What signup, verification, password reset, and recovery flows are needed?
3. What exact task states are needed? The draft/review-looking control's meaning and publication rules are unclear.
4. Are budgets fixed-price task totals, hourly rates, or estimates? What currencies and payment capabilities are in scope?
5. Can a task accept multiple applicants? What happens to other applications after acceptance? Can proposals be edited or withdrawn?
6. How are "matches" generated and what do accepted, pending, and other visible groups mean operationally?
7. What constitutes a verified profile, rating, completion percentage, earnings, or active task count?
8. What is the data model and update permission for the chat's project goal/progress card? Are read receipts, attachments, notifications, or realtime delivery required?
9. Should profile skills and portfolio highlights be snapshotted when an application is submitted?
10. What are retention, moderation/reporting, blocking, deletion, and audit requirements?

Recommended delivery sequence

Phase	Backend scope	Exit condition
1. Identity and profile	Supabase JWT verification, Bridge user mapping, user modes, profile CRUD, skill catalog.	A signed-in user can save/read their profile with enforced field permissions.
2. Marketplace	Task CRUD/status, browse/detail, application submit/list/decision.	An owner can publish and review applications; a student can browse and apply.
3. Match and messaging	Match/status list, conversation membership, message history/send.	Only authorized participants can access a conversation; decisions are consistent under retries/concurrency.
4. Dashboard metrics	Active tasks, pending payment display, ratings/verification aggregates.	Metric definitions and source data are agreed and values reconcile.

Source: Bridge: Student Talent Marketplace, Google Stitch project 14483777616722284649. Screen descriptions reflect visible UI content; API and schema sections are proposed backend contracts.